

LINFO2144 - Practical session 3:

Authenticated Key Exchange

In this practical session you will analyze several key exchange protocols.

Exercices some exercices are inspired by Boneh–Shoup’s book *A Graduate Course in Applied Cryptography* (version 0.6, Jan. 2023)¹.

One exercise is an old exam question (June 2025) and the last one is a practical attack on a vulnerable web server (an exercise from an 2025’s project).

1 Authenticated Key Exchange Variants

Inspired by Boneh–Shoup’s book. We study several variants of an authenticated key exchange protocol.

Tasks. For each AKE protocol described below:

- Describe the computations that have to be performed by the parties, and how the content of each message is used.
- Study if we have one-sided security, two-sided security, and/or forward secrecy. Justify this based on the properties of the cryptographic primitives used, or by describing an attack (for an attack, give the messages sent by all parties, and in case of messages sent by the adversary, justify that it can compute these values). Classify the attacks (key exposure, replay, identity misbinding, etc.).

Notations.

- $S = (S.Gen, S.Sign, S.Verify)$ denotes a secure digital signature scheme.
- $KEM = (KEM.Gen, KEM.Encap, KEM.Decap)$ denotes a secure key encapsulation mechanism.
- $(sk_P, pk_P) \leftarrow S.Gen()$ and $(sk_Q, pk_Q) \leftarrow S.Gen()$ denote the long-term signature key pairs of P and Q , respectively.

¹<https://toc.cryptobook.us/>

auth purpose, not encryption

- $(sk, pk) \leftarrow \text{KEM.Gen}()$ denotes a freshly generated (ephemeral) KEM key pair.
- $(k, c) \leftarrow \text{KEM.Encap}_{pk}()$ denotes encapsulation of a fresh session key k under public key pk , producing ciphertext c .
- $k \leftarrow \text{KEM.Decap}_{sk}(c)$ denotes decapsulation of ciphertext c using secret key sk .
- id_P and id_Q denote the public identities of P and Q .
- $\mathcal{K}$ denotes the set of session keys output by the protocol.

Protocol AKE1. P and Q each have a signature key sk_P, sk_Q . The other party knows the corresponding signature verification key pk_P, pk_Q .

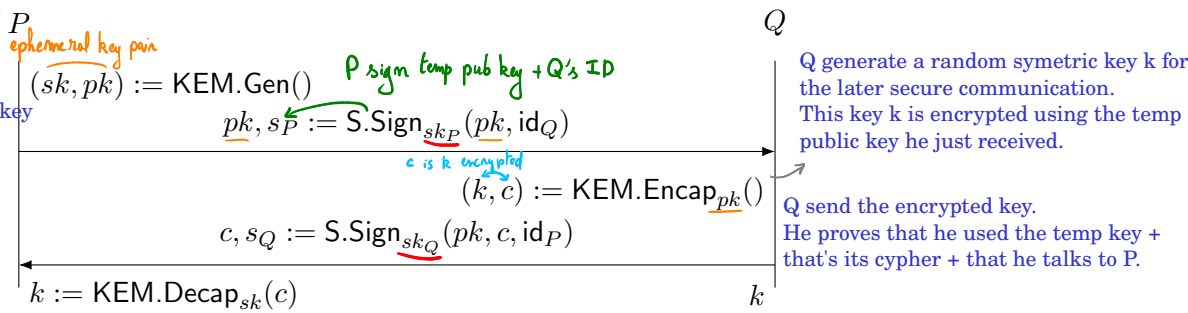
Long Term key pair to prove each other's identity

P say : I want to talk to Q.

Let's start a secure temporary channel, here is a temporary public key you can use to give me a symmetric key .

I prove it's me that wants to securely talk to Q by signing Q id and the temp key

P use the temporary secret key to get the temp symmetric key Q wants to use.



For AKE1, before analyzing its security properties, describe in a pseudocode the operations performed by P , and likewise for Q .

Key exposure

Variation 1 — Do not sign c . Same protocol as AKE1, except we have $s_Q := \text{S.Sign}_{sk_Q}(pk, id_P)$

Q do not sign the cypher containing the temp key to use for secure comm. If traffic sniff : attacker knows pk, so he can encrypt its own cypher.

Then he gave the cypher + prove that Q used the pk and wants to talk to P. P will send a message using modified k and the attacker can read it (not Q though).

Key exposure

Variation 2 — Q does not sign. Same protocol as AKE1, except s_Q is not sent.

Q receive info for a secure comm, but doesn't prove he's the one replying. As before, attacker will use the pk send the reply to P.

P will get the modified k and will reply using it. The attacker can read the secret messages P send to Q.

1 sided auth.

P is a client;

Q a server

Variation 3 — P does not sign. Same protocol as AKE1, except s_P is not sent.

Typical Client Server communication. The client said that he wants to talk to the server using this pk. If attacker modifies pk, generated server k won't be decrypted correctly. Client will send a message with a weird key and nobody can decrypt it.

Identity misbinding

Variation 4 — Do not sign id_P . Same protocol as AKE1, except we have $s_Q :=$

$\text{S.Sign}_{sk_Q}(pk, c)$ MITM with Eve : P thinks is talking to Q , while Q thinks he is talking to Eve. Eve won't know the content bc k remains secret.

Eve forward 1st packet to Q but strip P's signature. Q think Eve wants start to talk so generate a k using the pk. Eve get the responds and forward it to P.

No forward Secrecy

Variation 5 — Stable KEM keys. Same protocol as AKE1, (sk, pk) is part of the initial state of P , and the same values are used for all its executions.

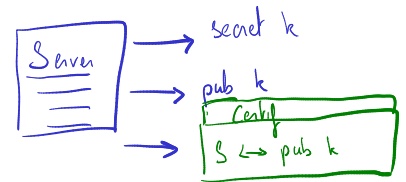
Here we just loose forward secrecy. If those key leak, all previous communication can be decrypted. Previously it was always new ephemeral pairs that got discarded after the current communication was over.

Variation 6 — identities in KDF. Same protocol as AKE1, except we have $s_P :=$

$\text{S.Sign}_{sk_P}(pk)$ $s_Q := \text{S.Sign}_{sk_Q}(pk, c)$, and both parties compute their output as $k' :=$

$F(k, (id_P, id_Q))$ where F is a hash function.

This works only if both id_P and id_Q are shared secret.



2 Key establishment (June 2025 exam question)

Given a KEM ($\text{KEM.Gen}, \text{KEM.Encap}, \text{KEM.Decap}$), let us consider the following key establishment protocols between a client and a server. For both protocols the server S holds a KEM keypair (generated as $(sk_S, pk_S) \xleftarrow{\$} \text{KEM.Gen}()$) and a certificate over the pair (S, pk_S) . We assume that the client can verify the authenticity of the certificate.

- KE1:

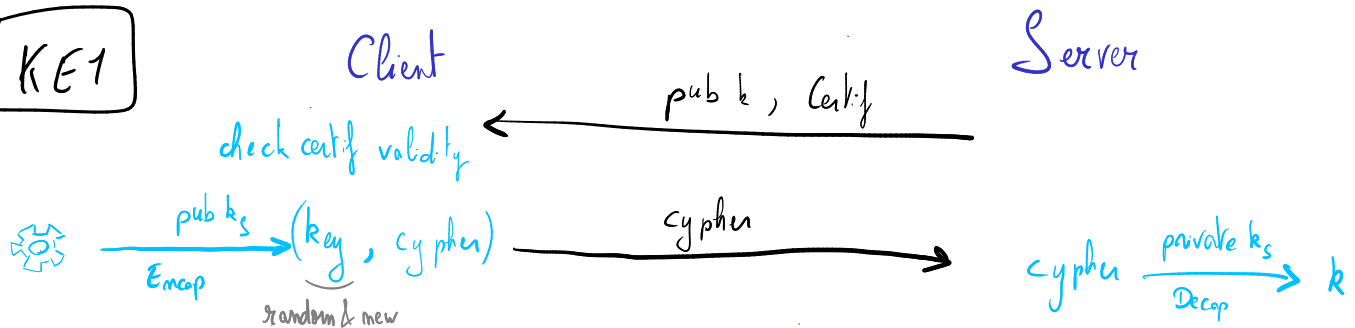
1. The client C opens a connection to S .
2. S sends pk_S and its certificate to C .
3. C , upon receiving a public key pk and a certificate, checks that the certificate is valid for (S, pk) (if not, it aborts).
4. C computes $(k_C, c_C) \xleftarrow{\$} \text{KEM.Encap}_{pk_S}()$ and sends c_C to S .
5. S , upon receiving a ciphertext c_S , computes $k_S \leftarrow \text{KEM.Decap}_{sk_S}(c_S)$
6. The result of the protocol for C is k_C and the result for S is k_S .

- KE2:

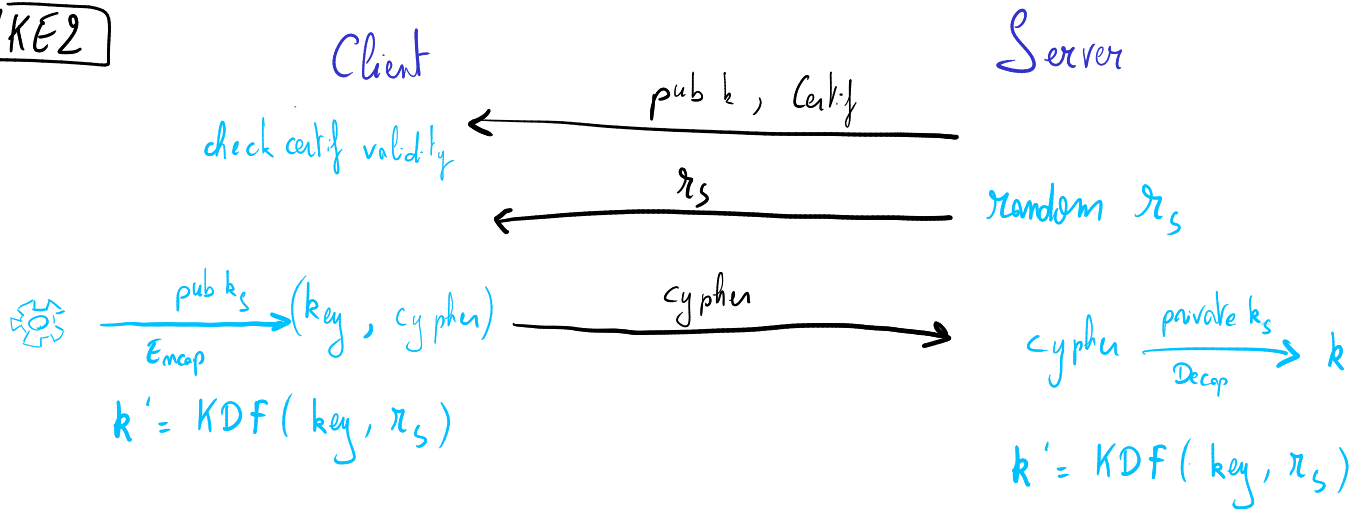
1. The client C opens a connection to S .
2. S sends pk_S and its certificate to C .
3. C , upon receiving a public key pk and a certificate, checks that the certificate is valid for (S, pk) (if not, it aborts).
4. S **samples** $r_S \xleftarrow{\$} \{0, 1\}^{256}$ **and sends it to** C .
5. C **receives** r_C **from** S .
6. C computes $(k_C, c_C) \xleftarrow{\$} \text{KEM.Encap}_{pk}()$ and sends c_C to S .
7. S , upon receiving a ciphertext c_S , computes $k_S \leftarrow \text{KEM.Decap}_{sk_S}(c_S)$
8. **The result of the protocol for** C **is** $k'_C = \text{KDF}(r_C, k_C)$ **and the result for** S **is** $k_S = \text{KDF}(r_S, k_S)$ (where KDF is a secure key derivation function).

- If both parties execute the protocol correctly and the communication between them is not tampered with, what are the important properties satisfied by the outputs of these key establishment protocols?
- What is the weakness in KE1 that is fixed in KE2? How could it be exploited in practice (e.g., you may consider that this is the key establishment for a TLS-like protocol) ?
- Which security property/properties does the TLS key establishment enjoy, compared to KE2? Propose a way to improve KE2 in that respect, without using a digital signature **Sign** operation in the protocol.

KE1



KE2



1) Properties for both KE :

- Correctness : both client and server will get the same shared secret session key.
- Confidentiality : only the client and the server will know the session key.
- One sided authentication : The client is guaranteed that they are establishing a key with the legitimate server S (because only S holds the private sk_S matching the verified certificate). However, the server does not know who the client is.

2) weakness of KE1 improved by KE2 :

- **The Weakness (Lack of Server Contribution/Freshness):** In KE1, the final session key is dictated entirely by the client (k_C). The server contributes no randomness. Because the server's public key (pk_S) is static, if an attacker records the client's ciphertext (c_C), they can replay it at any time in the future.
- **The Fix in KE2:** KE2 introduces a random nonce generated by the server ($r_S \leftarrow \{0, 1\}^{256}$). The final session key is derived using a Key Derivation Function ($KDF(r_S, k_C)$). This ensures that the server contributes to the **freshness** of the session key.
- **Exploitation in Practice (Replay Attack):**
If KE1 is used for a TLS-like protocol, an attacker can record the initial key exchange (c_C) and the subsequent encrypted application data (e.g., an encrypted HTTP POST /transfer?amount=100 request).
Later, the attacker opens a new connection to S and replays c_C . The server will decapsulate c_C and derive the *exact same session key* k_S as before. The attacker then replays the encrypted HTTP request. The server will successfully decrypt it and process the bank transfer a second time.
In KE2, this attack fails: if the attacker replays c_C , the server will generate a *new, completely different* r_S for the session. The resulting key $KDF(r_S, k_C)$ will not match the old key, and the replayed encrypted HTTP request will fail the AEAD decryption check (outputting $\perp$).

Question 3: TLS comparison and Improving KE2

Which security property/properties does the TLS key establishment enjoy, compared to KE2? Propose a way to improve KE2 in that respect, without using a digital signature *Sign* operation in the protocol.

Solution:

- **Missing Security Property:** As you correctly noted, the TLS key establishment enjoys **Perfect Forward Secrecy (PFS)**, which KE2 entirely lacks. In KE2, if the server's long-term private key (sk_S) is compromised in the future, an attacker can look at their logs, decapsulate past c_C values to get k_C , and easily compute $KDF(r_S, k_C)$ because r_S was sent in plaintext.
- **How to improve KE2 (Achieving Forward Secrecy without *Sign*):**
To achieve Forward Secrecy, we must use an *ephemeral* key exchange, but we are not allowed to use Digital Signatures to authenticate it. We can solve this by encapsulating against **two** different public keys: the server's static certificate key (for authentication) and a newly generated ephemeral key (for forward secrecy). This is heavily inspired by the **KEMTLS** framework.

Improved Protocol (KE3):

1. C opens a connection to S .
2. S generates a fresh, **ephemeral** KEM keypair: $(sk_E, pk_E) \leftarrow KEM.Gen()$.
3. S sends to C : its static certificate (containing static pk_S) **and** the ephemeral public key pk_E .
4. C verifies the certificate for (S, pk_S) .
5. C performs **two** encapsulations:

- $(k_{stat}, c_{stat}) \leftarrow KEM.Encap(pk_S)$ (Authenticates the server)
- $(k_{eph}, c_{eph}) \leftarrow KEM.Encap(pk_E)$ (Provides Forward Secrecy)

6. C sends both ciphertexts (c_{stat}, c_{eph}) to S .

7. S decapsulates both:

- $k_{stat} \leftarrow KEM.Decap(sk_S, c_{stat})$
- $k_{eph} \leftarrow KEM.Decap(sk_E, c_{eph})$

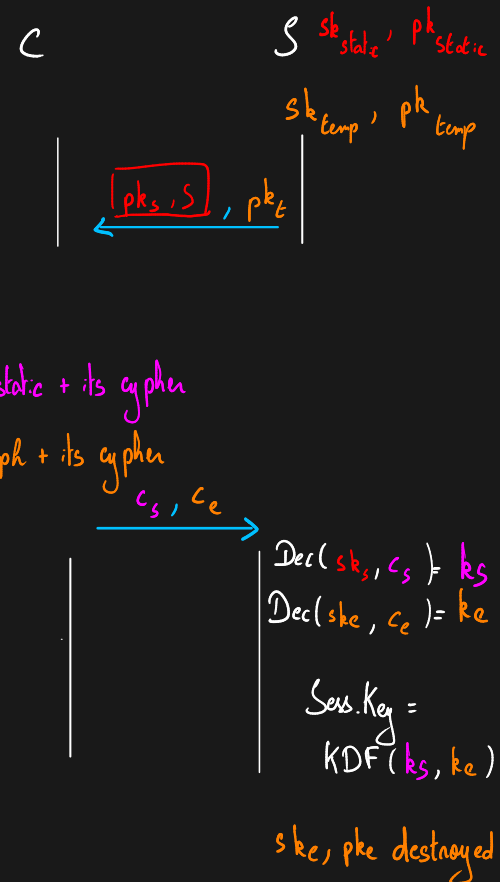
8. Both parties compute the final session key using the KDF:

$$Key = KDF(k_{stat}, k_{eph}).$$

9. S instantly deletes sk_E . ←

Why this works:

- **No *Sign* operation is used:** Authentication is achieved implicitly because only the true Server possesses sk_S to decapsulate c_{stat} .
- **Forward Secrecy is achieved:** The final session key is bound to k_{eph} . Because sk_E is ephemeral and deleted immediately after the handshake, an attacker who compromises the static sk_S years later cannot decapsulate c_{eph} , meaning they cannot compute the KDF to recover the final session key.



3 Dummy Key Exchange Protocol

Exercise 10.2 in Boneh-Shoup's book.

Let us consider a key exchange protocol $\mathcal{P}$ between two parties P and Q , whose output key is k . An eavesdropping adversary A is a restricted adversary who can observe all messages sent by P and Q (the *transcript* T of the protocol execution). Its goal is to guess k : its outputs $\hat{k}$ and succeeds if $\hat{k} = k$. We define A 's advantage as the probability that $\hat{k} = k$: $\text{AnonKEadv}[A, \mathcal{P}] = \Pr[\hat{k} = k \mid \hat{k} \leftarrow A(T)]$.

Assume that P does not sample randomness in the execution of the protocol and does not have secret values as part of its initial state. Show that, in such a case, $\mathcal{P}$ cannot be secure against eavesdroppers. In particular, build A such that $\text{AnonKEadv}[A, \mathcal{P}] = 1$.

4 BREACH attack

For this exercise, you will take advantage of the BREACH attack².

4.1 How to run the exercise

You do not need the VM but Docker and python3. To build the docker, run the command `docker build . -t breach` inside the directory BREACH. To run it, use `docker run -d -p 5000:5000 -p 5001:5001 breach`. Do not forget to kill your container when you have finished using it with `docker kill <container_name>`.

4.2 Explanation of the setup

At the address `https://localhost:5000` runs a server with the following pages:

1. At `/` you have a home page
2. At `/signin` you can connect to an existing account
3. At `/signup` you can create an account
4. At `/logout` you can logout from your session
5. At `/secret?username=<username>` you can have a link to retrieve the flag
6. At `/get_flag?csrf_token=<csrf_token>` you have the flag if you have the CSRF token of the oracle. This page is only to check if you found the right CSRF token. Normally, you need to be connected as the oracle to have access to the content. The CSRF token is not normally used as a connection token.

At the address `http://localhost:5001` runs an oracle. This oracle simulates a client over which you have some amount of control (through a technical or social engineering attack). It is connected to the server with its credentials and can make requests to it. On

²<https://en.wikipedia.org/wiki/BREACH>

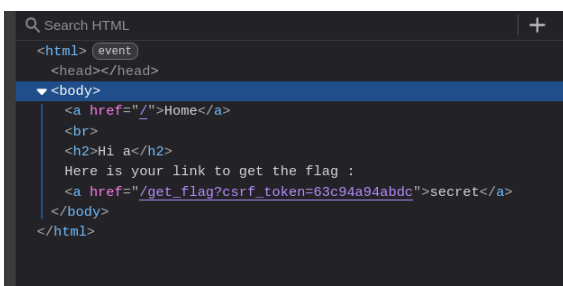
the home page (/), you have a form where you can submit a request string. The oracle will make the request to the server. For example, if you request `get_flag`, the oracle will request `https://localhost:5000/get_flag`. Then, the oracle will respond with the content length of the response received from the server, simulating what a network adversary could observe. The file `breach_attack.py` shows an example of request to the oracle and prints the response.

4.3 Task

Your goal is to find the secret CSRF token of the oracle and request the flag to the server with it. The CSRF token will always have the same size and only contains lowercase hexadecimal characters. A simple brute force over the CSRF token without using the oracle will not be accepted as a solution. To help you get started, here are a few questions to answer. (They can be answered with one or two sentences.)

1. What is the vulnerability used in BREACH ? [http compression of repeated strings](#)
2. How can you find (without looking at the code of the server) the length of the CSRF token assuming they have a fixed length ? [create an account and check html](#)
3. Which page of the server can be used to take advantage of the vulnerability and why ? [at /secret you can but as argument your guess and the response contains the guess + csrf -> so we can analyse for compression](#)
4. How the content length provided by the oracle can be useful ? [if answer smaller means there is a match](#)
5. Which requests will you make to the oracle to retrieve the CSRF token ? [/secret?username=myguess](#)
6. The oracle returns the HTTP **Content-Length**. How could you observe this as an adversary? [wireshark and measure the length of the packet ?](#)
7. (To be answered after making the attack) Which TLS feature could prevent your attack? (BONUS: How could you modify your attack to bypass this mitigation?)
8. (BONUS - May be answered after making the attack) This exercise stops after recovering the CSRF token. In an actual attack, how could you exploit knowledge of the token?
9. (BONUS - May be answered after making the attack) An element in the server is misused. How can you modify the server so that it no longer has this vulnerability but still supports compression?

It is a side-channel attack that allows an attacker to extract secrets from encrypted HTTPS traffic. Normally, HTTPS (TLS/SSL) encrypts the contents of a web page so an attacker snooping on the network only sees garbage data. However, they can still see the size (length) of the encrypted data being sent back and forth.



```
Search HTML
<html> (event)
<head> </head>
<body>
  <a href="/">Home</a>
  <br>
  <h2>Hi</h2>
  Here is your link to get the flag :
  <a href="/get_flag?csrf_token=63c94a94abdc">secret</a>
</body>
</html>
```